

Стандарт разработки на C++

Имена файлов и папок

Файлы и папки именуются строчными латинскими буквами (doxygen не принимает русские имена). Для разделения слов в именах файлов и папок используется подчеркивание (так ближе к C++, например, `shared_ptr.hpp`). Одно исключение – в именах исполняемых файлов в качестве разделителя используется дефис (например, `sbis-lib300.dll`, `sbis-rpc-worker300.exe` и т.д.).

Расширения файлов

Для заголовочных файлов используется расширение `".h"` или `".hpp"`, для файлов с кодом – `".cpp"`.

Организация папки проекта

Все файлы, относящиеся к проекту, лежат внутри папки названной по имени проекта. Эта папка является **корневой папкой проекта**. Внутри нее должны быть две обязательные папки:

1. папка экспортируемых проектом интерфейсов (если есть, что экспортировать);
2. папка реализации проекта.

Также в корневой директории проекта должен присутствовать файл описания проекта **CMakeLists.txt** в кодировке UTF-8 (без BOM).

Дополнительно в корневом каталоге проекта могут быть следующие каталоги:

1. `libs` – каталог, в котором в скомпилированном виде лежат сторонние библиотеки, необходимые для сборки данного проекта.
2. `res` – дополнительные ресурсы, которые должны быть скопированы в каталог сборки вместе с программным артефактом (библиотекой, исполняемым файлом, модулем), производимым проектом.

Каталог экспортируемых сущностей проекта

Эта папка называется по имени проекта. В ней описывается экспортируемый программный интерфейс проекта (API). В этой папке должны храниться **исключительно** экспортируемые заголовочные файлы, в которых должны быть объявлены только экспортируемые сущности. Все остальные не экспортируемые сущности хранятся в папке реализации.

Название папки по имени проекта позволяет удобно делать вставку заголовочных файлов в других проектах. Например, если проект называется `"sbis-dbi"`, то вставка заголовочного файла `"idatabase.hpp"` будет выглядеть так:

```
#include <sbis-dbi/idatabase.hpp>
```

Каталог реализации проекта implementation

В папке с именем `"implementation"` (реализация) находятся `cpp`-файлы и внутренние заголовочные файлы. У больших проектов папка `"implementation"` должна быть разбита на подпапки по функциональным подсистемам проекта.

В корне папки `"implementation"` должны находиться только файлы, которые нельзя отнести ни к одной из подсистем. Такими файлами для нас являются:

1. файлы для создания предкомпилируемых заголовков `precomp.h` и `precomp.cpp`;
2. файл `"dllmain.cpp"`, хранящий реализацию функции `DllMain`/конструктора разделяемой библиотеки.

Каталог сторонних библиотек libs

Зачастую при разработке наших решений возникает необходимость использовать сторонние библиотеки. В этом случае встает вопрос, как эти библиотеки подключить к своему проекту. Для решения этой задачи в платформенном

модуле sbis.cmake разработана специальная директива sbis_use_library со следующим синтаксисом:

```
sbis_use_library( имя_библиотеки [дополнительные параметры] )
```

Если никакие дополнительные параметры не указаны, то поиск библиотеки производится в подкаталоге libs корневого каталога того проекта, который "заказал" ее подключение. Данный каталог должен быть организован следующим образом:

```
libs
├── имя_библиотеки1                // Все, что относится к
имя_библиотеки1.
│   ├── include                  // Заголовочные файлы библиотеки.
│   └── lib                      // Скомпилированные версии
библиотеки.
│   ├── OC1                     // Версии библиотеки под
операционную систему "OC1".
│   │   ├── архитектура1       // Версии библиотеки под
архитектуру "архитектура 1".
│   │   │   ├── debug          // Отладочная версия библиотеки.
│   │   │   └── release        // Релизная версия библиотеки.
│   │   └── архитектура2
│   │       ├── debug
│   │       └── release
│   └── OC2
│       └── архитектура1
│           ├── debug
│           └── release
└── имя_библиотеки2
    ├── include
    └── lib
```

Таким образом, один подкаталог каталога libs соответствует одной подключаемой к проекту сторонней библиотеке (подкаталог должен называться так же, как и библиотека в директиве sbis_use_library).

В каждом таком подкаталоге должны присутствовать два вложенных каталога:

1. include – заголовочные файлы подключаемой библиотеки.
2. lib – скомпилированные варианты данной библиотеки для каждой комбинации (Операционная система, архитектура, релиз/отладка). Структура данного каталога организуется соответствующим образом.

Допускается помещать релизные сборки библиотек (если runtime позволяет, конечно же) в каталоги для отладочных версий, если отладка кода сторонней библиотеки не требуется.

Все сторонние библиотеки, используемые в проекте, должны быть собраны с отладочной информацией. Символы к ним должны быть публично доступны.

Дополнительные ресурсы проекта – res

Некоторым проектам для корректной работы требуются дополнительные ресурсы. Это могут быть какие-то сторонние утилиты, файлы и т.п. Зачастую, такие файлы являются платформозависимыми. Для того чтобы обеспечить попадание подобных файлов в каталог сборки, необходимо в корневом каталоге проекта создать подкаталог res, организованный по тому же принципу, что и каталог libs/библиотека/lib из предыдущего раздела. Тогда все, что попадет под текущую комбинацию (операционная система, архитектура, релиз/отладка), будет скопировано как дополнительные продукты сборки в сборочный каталог.

Обеспечение интеграции проекта в приложение компании

Почти все проекты на C++ должны быть упакованы в модуль бизнес-логики в той или иной роли: главная библиотека, тест или дополнительный ресурс.

В разделе Sources должна быть задана ссылка на папку проекта. Подробно описано в статье [Структура проекта приложения](#).

Организация кода

#pragma once

Для предотвращения повторного включения заголовочных файлов должна использоваться директива компилятора #pragma once, а не include guards.

```
1 // Плохо
2 #ifndef SOME_LONG_UNCLUDE_GUARD
3 #define SOME_LONG_UNCLUDE_GUARD
4
5 void foo( int );
6
7 #endif
8
9 // Хорошо
10 #pragma once
11
12 void foo( int );
```



#pragma once — это специализированное средство для предотвращения повторного включения заголовочных файлов. Использование include guard — это "костыль".

Организация заголовочных файлов проекта

Одним из важнейших факторов, влияющих на скорость и удобство разработки на языке C++, является время (пере-)сборки проекта. Это время, в свою очередь, весьма существенно зависит от того, каким образом в программе организованы зависимости между ее компонентами. При аккуратном управлении зависимостями между заголовочными файлами можно достичь кратного ускорения компиляции.

Все заголовочные файлы проекта (и экспортируемые, и внутренние) должны удовлетворять двум критериям:

1. минимальность
2. самодостаточность

Под минимальностью понимается следующее: данный заголовочный файл должен подключать в себя лишь те файлы, без которых он не может быть использован. Т.е. не должно быть лишних #include.

Под самодостаточностью понимается следующее: все символы, используемые в данном заголовочном файле, должны быть в каком-то виде в нем описаны.

Правила для достижения самодостаточности

1. После подключения любого заголовочного файла в чистый cpp-файл этот файл должен компилироваться без ошибок. Также для всех типов, непосредственно описанных в подключенном заголовочном файле, должен успешно обрабатывать оператор sizeof().
2. Если в заголовочном файле описан класс Foo, унаследованный от класса Bar, то данный заголовочный файл должен самостоятельно подключать файл, в котором описан Bar.

```
1 // Плохо
2 #pragma once
3 class Foo : public Bar
4 {};
5
6 // Хорошо
7 #pragma once
8 #include <bar.hpp>
9 class Foo : public Bar
10 {};
```

3. Если в заголовочном файле описан класс Foo, одним из членов которого является экземпляр класса Bar, то данный заголовочный файл должен самостоятельно подключать файл, в котором описан Bar.

```
1 // Плохо
2 #pragma once
3 class Foo
4 {
5     Bar mBar;
6 };
1 // Хорошо
2 #pragma once
3 #include <bar.hpp>
4 class Foo
5 {
6     Bar mBar;
7 };
```

4. Если в заголовочном файле описан шаблон функции или класса, то все заголовочные файлы, объявляющие типы, макросы и функции, необходимые для компиляции шаблона, должны быть подключены в данный заголовочный файл. Этот пункт не относится к типам, которые пользователь намеревается использовать в качестве параметров шаблона.

```
1 // Плохо
2 #pragma once
3 template< typename T >
4 void Foo()
5 {
6     Bar< T > b;
7     ProcessBar( b );
8 }
1 // Хорошо
2 #pragma once
3 #include <bar.hpp>
4 #include <process_bar.hpp>
5 template< typename T >
6 void Foo()
7 {
8     Bar< T > b;
9     ProcessBar( b );
10 }
```

5. Для каждого конкретного заголовочного файла ответственность за подключение всех его зависимостей лежит на авторе данного файла. Т.е. нельзя полагаться на транзитивные #include.

```
1 // Плохо
2 // foo.hpp
3 #pragma once
4 class Foo
5 {};
6 // bar.hpp
7 #pragma once
8 #include <foo.hpp>
9 class Bar
10 {
11     Foo mFoo;
12 };
13 // baz.hpp
14 #pragma once
15 #include <bar.hpp>
```

```

16 class Baz
17 {
18     Foo mFoo;
19     Bar mBar;
20 };
1 // Хоршо
2 // foo.hpp
3 #pragma once
4 class Foo
5 {};
6 // bar.hpp
7 #pragma once
8 #include <foo.hpp>
9 class Bar
10 {
11     Foo mFoo;
12 };
13 // baz.hpp
14 #pragma once
15 #include <foo.hpp>
16 #include <bar.hpp>
17 class Baz
18 {
19     Foo mFoo;
20     Bar mBar;
21 };

```

Правила для достижения минимальности

1. Если какой-то символ в заголовочном файле может остаться только в виде forward-declaration, то его **необходимо использовать** именно в таком виде. Примерами таких символов являются типы аргументов и возвращаемых значений методов.

```

1 #pragma once
2 // forward-declaration используемых типов.
3 class Foo;
4 class Bar;
5
6 // Для подключения этого файла не нужно видеть полное описание Foo и
  Bar.
7 Foo GetFoo( Bar const& bar );

```

2. Если какой-то тип часто используется в виде forward-declaration и при этом его предварительное объявление не тривиально (например, специфический вариант инстанцирования шаблонного класса), то имеет смысл сделать отдельный заголовочный файл, содержащий такой forward-declaration.

```

1 #pragma once
2 // forward-declaration шаблона.
3 template< typename T >
4 class Foo;
5
6 // forward-declaration для typedef'a.
7 typedef Foo<int> Bar;

```

3. Необходимо минимизировать количество кода в заголовочных файлах. Вся логика функций/классов должна реализовываться в src-файлах. Исключение составляют только совсем простые методы в 1-2 строки и шаблоны.
4. Необходимо минимизировать использование публичных вложенных классов (для них нельзя сделать forward-declaration). Если нужно ограничить область видимости некоторого типа, лучше использовать вложенное пространство имен.

```

1 // Плохо
2 #pragma once
3 class Foo
4 {
5     public:
6         class Bar
7         {};
8 };
1 // Хорошо
2 #pragma once
3 namespace foo
4 {
5     class Bar
6     {};
7 }
8 class Foo
9 {};

```

5. Заголовочные файлы проекта (особенно экспортируемые) должны проектироваться таким образом, чтобы максимально гранулярно описывать поставляемый функционал. Запрещается делать заголовочные файлы вида "все в одном". В идеале необходимо стремиться к стратегии "один класс – один файл". Следование подобной стратегии обеспечивает минимальное время пересборки зависимостей при внесении изменений в код.

```

1 // Плохо
2 // all_in_one.hpp
3 #pragma once
4 #include <dependency1.hpp>
5 #include <dependency2.hpp>
6 #include <dependency3.hpp>
7 class Foo
8 {};
9 class Bar
10 {};
11 class Baz
12 {};
1 // Хорошо
2 // foo.hpp
3 #pragma once
4 #include <dependency1.hpp>
5 class Foo
6 {};
7 // bar.hpp
8 #pragma once
9 #include <dependency2.hpp>
10 class Bar
11 {};
12 // baz.hpp
13 #pragma once
14 #include <dependency3.hpp>
15 class Baz
16 {};

```

6. Если ваш класс в качестве членов содержит "умные" указатели на тип T, и для других целей информация о типе T в данном заголовочном файле не требуется, то имеет смысл определить в классе деструктор и вынести его реализацию в сpp-файл.

```

1 // Плохо
2 // baz.hpp

```

```

3  #pragma once
4  #include <memory>
5  #include <foo.hpp>
6  #include <bar.hpp>
7  class Baz
8  {
9      private:
10         std::unique_ptr< Foo > mFoo;
11         std::shared_ptr< Bar > mBar;
12 };

1 // Хоршо
2 // baz.hpp
3 #pragma once
4 #include <memory>
5 class Foo;
6 class Bar;
7 class Baz
8 {
9     public:
10         ~Baz();
11     private:
12         std::unique_ptr< Foo > mFoo;
13         std::shared_ptr< Bar > mBar;
14 };
15 // baz.cpp
16 #include <foo.hpp>
17 #include <bar.hpp>
18 #include <baz.hpp>
19 Baz::~~Baz()
20 {}

```

Прекомпилируемые заголовочные файлы

В проектах на Wasaby Framework могут использоваться прекомпилируемые заголовочные файлы. Такой файл (если он есть) должен располагаться в корне каталога implementation и называться presomp.h и никак иначе. В таком файле не должна использоваться директива #pragma once (GCC в этом случае выдает предупреждение). Подключение прекомпилируемого заголовочного файла к проекту осуществляется директивой sbis_add_precompiled_header() модуля sbis.cmake.

К содержимому прекомпилируемого заголовочного файла необходимо относиться крайне аккуратно. С одной стороны, данная техника ускоряет компиляцию за счет повторного использования ранее скомпилированных элементов. С другой стороны, при использовании прекомпилируемых заголовочных файлов растет потребление компилятором ресурсов и размер отладочной информации. Поэтому количество подключенных в presomp заголовочных файлов необходимо минимизировать.

Запрещается:

1. Подключать в прекомпилируемый заголовок проекта заголовочные данные из самого проекта.
2. Подключать в прекомпилируемый заголовок проекта "весь STL" или "весь Boost".

Стандарт языка

При написании кода необходимо придерживаться общего подмножества возможностей, реализованных в GCC 4.7 и Visual Studio 2010. Некоторые правила.

Использование nullptr

Во всех случаях при написании кода необходимо использовать ключевое слово nullptr. Использование макроса NULL не допускается.

Использование ключевого слова `auto`

Ключевое слово `auto` можно использовать только в том случае, если получаемый тип выражения очевиден из выполняемого присваивания.

```
1 // Плохо
2 // Не ясно, что возвращает DoSomeCoolStuff.
3 auto foo = DoSomeCoolStuff();

1 // Хорошо
2 // Тип результата очевиден.
3 auto foo = std::make_shared< Foo >();
4
5 // Стандартная нотация получения итератора.
6 auto bar = mContainer.begin();
```

Использование `trailing return type syntax`

Запрещается использовать `trailing return type syntax` за исключением случая вывода возвращаемого типа шаблонной функции (в сочетании с `decltype`) в том случае, когда он не очевиден.

```
1 // Плохо
2 auto foo( int ) -> bool;
3 template< typename T >
4 auto bar( T ) -> T;

1 // Хорошо
2 template< typename U, typename V >
3 auto foo( U u, V v ) -> decltype( u + v );
```

Использование ключевого слова `override`

При перекрытии в классе-наследнике виртуального метода класса-предка необходимо использовать ключевое слово `override`.

```
1 // Плохо
2 class Foo
3 {
4     virtual void DoSomething();
5 };
6 class Bar : public Foo
7 {
8     void DoSomething();
9 };

1 // Хорошо
2 class Foo
3 {
4     virtual void DoSomething();
5 };
6 class Bar : public Foo
7 {
8     virtual void DoSomething() override;
9 };
```

Устаревшие функции, классы и возможности языка

При разработке запрещено пользоваться устаревшими функциями и классами стандартной библиотеки, а именно:

1. `std::auto_ptr<>` — используйте `std::weak_ptr<>`

2. `std::bind1st`, `std::bind2nd`, `std::unzipy_function` и т.д. – используйте лямбда-функции или `std::bind`

Также не следует при написании нового кода использовать `Dynamic exception specification` – оно просто не нужно.

Единственное исключение — наследование от устаревших классов, которые его используют.

Синтаксис

Отступ (табуляция)

Величина отступа (табуляции) – **3 (три)** знакоместа, то есть пробела. Причём ставятся именно пробелы, а не символ табуляции.

Фигурные скобки – {}

Открывающая фигурная скобка ставится на новой строке без отступа. Текст после скобки начинается на следующей строке с отступом. Закрывающая фигурная скобка ставится ровно под открывающей.

```
1 // Хорошо
2 if( a > b )
3 {
4     a = b;
5 }

1 // Плохо
2 if( a>b ) { a = b; b = c; }
3
4 if( a>b )
5 { a = b; b = c; }
6
7 int DoIt()
8 { a = b; }
9
10 int DoIt()
11 { a = b;
12  b = c;
13 }
14 do
15
16 {
17 }while(...)
```

Круглые скобки – ()

Круглые скобки после `"if"`, `"for"` и тому подобное ставятся без пробела. Например:

```
for( int i = 0; i < n; i++ )
```

Выражения

Присваивание **всегда** обрамляется с обеих сторон пробелами. Каждое выражение должно начинаться с новой строки. Например:

```
1 int a = b + c;
2 x = z - y;
3 db.Open();
```

При использовании оператора `"?:"` обязательно должны быть поставлены пробелы:

```
x = a < b ? a : b;
```

Не допускается использование оператора "?:" как замены для if:

```
a < b ? a = 10 : b = 20;
```

Операторы

1. if else

Для описания используется конструкция:

```
1 // Комментарий, если требуется.
2 if( a < b )
3 {
4     // Необходимые действия.
5 }
6 // Комментарий, если требуется.
7 else if( d > c )
8 {
9     // Необходимые действия.
10 }
11 // Комментарий, если требуется.
12 else
13 {
14     // Необходимые действия.
15 }
```

Возможен вариант:

```
1 // Комментарий, если требуется
2 if( a < b ) // Действие (одно!)
3
4 // Комментарий, если требуется
5 else if( d > c ) // Действие (одно!)
6 // и т.д.
```

2. switch case

```
1 switch( параметр )
2 {
3     case 1:
4         // Необходимые действия
5         break;
6     case 2:
7         // Необходимые действия
8         break;
9     default:
10        // Необходимые действия
11 }
```



Если в одном из случаев Вы умышленно не написали break, то в комментарии необходимо об этом написать и указать причину!

Альтернативный синтаксис операторов

Запрещается использовать [альтернативный синтаксис операторов](#).

Имена переменных и функций

1. Все имена должны отвечать правилам английского языка. **Русские слова в латинской транслитерации не допускаются!** Если с английским вы не очень, обратитесь к словарю.
2. Для стандартных действий рекомендуется использовать следующие имена:
Ins, Del, Repl, Remove, Get, Set, Count, Load, Store, Clear, Find, Write, Read, Init, Build, Test.
3. Функции, возвращающие значения опций, состояний, размеров должны именоваться без префикса **Get**.
Например: int Size(), int Options().
4. Функции, устанавливающие значения опций, состояний, размеров должны именоваться с префиксом **Set**.
Например: SetSize(int), SetOptions(int).
5. Используйте forward-declaration для перечислимых типов (при необходимости указывая underlying type). Ранее при объявлении перечислимых типов применялся специальный макрос ENUM_DECL(EnumName) из sbis-lib/defines.hpp. В данный момент времени в его применении в новом коде нет никакой необходимости, его возможности полностью обеспечиваются конструкцией enum class.

```
1 // foo.hpp
2 #pragma once
3 enum class Foo
4 {
5     ONE = 0,
6     TWO = 1
7 };
8 // bar.hpp
9 #pragma once
10 // forward-declaration перечислимого типа
11 enum class Foo;
12 Foo GetFoo();
```

6. При именовании переменных и функций следует придерживаться следующих правил написания имени:
 - Глобальные переменные, все функции, методы классов, типы данных, классы – все слова в имени должны начинаться с буквы в верхнем регистре, остальные буквы в нижнем регистре, подчёркивание не используется. Например: MainDB, Buf, DataView, GetPos.
 - Поля структур и классов должны начинаться с префикса m (от англ. "member"), чтобы понимать с чем мы работаем - с членом класса или локальной переменной. Например: mData, mBuf, mMainMenu.
 - Параметры функций, локальные переменные – все буквы в имени должны быть в нижнем регистре, для разделения слов использовать подчёркивание. Например: refresh_rec, i, str, dst_buf.
 - Имена глобальных констант должны быть в верхнем регистре, слова в имени разделяются подчёркиванием. Например: SMALL_SPLITTER_NAME.
7. Не рекомендуется именовать переменные с отрицанием или отрицательным смыслом.

```
1 // Плохо
2 bool is_disabled = true;
3 bool is_unmuted = false;
4 bool is_not_zero = true;
5
6 // Хорошо
7 bool is_enabled = false;
8 bool is_muted = true;
9 bool is_zero = false;
```

Описания функций и методов классов

При описании функций и методов классов не допускается опускать имена параметров, если не очевидно, что этот параметр означает. Например, так писать **нельзя**:

```
int SetParam(int,int,int);
```

Нужно писать так:

```
int SetParam( int x, int y, int size );
```

или:

```
int Remove( Record& );
```

Функции, которым в качестве параметра передаются два параметра типа "Откуда" и "Куда", должны первым параметром принимать параметр "Куда", а вторым – "Откуда". Точно так же, как библиотечные функции memcpy, strcpy и т. д. То есть по аналогии с оператором присваивания, где сначала указывается куда, а потом что. Например:

```
int CopyFlds( Record& dst, const Record& src, FieldType );
```

Описание пространств имен

1. **Каждый модуль** должен иметь **одно** свое экспортируемое пространство имен, вложенное в пространство имен sbis.
2. Внутри модуля можно заводить необходимое количество пространств имен, но **экспортировать их нельзя**.
3. Пространства имен называются **строчными символами**, желательно кратко (не более 7-ми символов) и, отражая суть модуля.

Пример:

```
1 dbi – database interface,  
2 bl – business logic  
3 rpc – remote procedure call  
4 и т.п.
```

Описания классов

1. В описании класса сначала должен идти раздел public, далее раздел protected, а потом private. Внутри каждого раздела должны быть сначала описаны поля, а потом методы класса.
2. Перед словами "protected" и "private", а также после них должна быть оставлена пустая строка. Пустая строка должна быть и между полями и методами класса.
3. Все члены классов выравниваются по левому краю.
4. Перечисление "друзей" класса идёт самым последним перед закрывающей скобкой.
5. Если класс переопределяет наследуемый виртуальный метод, то после сигнатуры метода обязательно нужно указать "override".

Пример описания класса:

```
1 class Foo  
2 {  
3     public:  
4  
5         explicit Foo( Bar bar );  
6         virtual ~Foo();  
7  
8         virtual void DoSomething() = 0;  
9         virtual Baz* DoSomethingElse( Data& );
```

```

10         virtual bool GetData( Data& );
11
12     protected:
13
14         Data mData;
15         int mValue;
16
17         void DoInternalStuff( int value );
18
19     private:
20
21         double mPrivateValue;
22
23         void DoPrivateStuff();
24
25         friend class Bar;
26     };

```

```

1 class Derived : public Foo
2 {
3     typedef Foo MyParent;
4
5     public:
6
7         explicit Derived( Bar bar );
8         virtual ~ Derived();
9
10        virtual void DoSomething() override;
11 };

```

Описания перечислимых типов

Все используемые в коде перечислимые типы (enums) должны быть строго типизированными, т.е. должен применяться синтаксис `enum class`. Использование данной возможности языка предотвращает "загрязнение" окружающего пространства имен и нежелательные неявные преобразования в/из целочисленных типов. При необходимости для перечислимого типа может указываться `underlying type`.



Ранее при объявлении перечислимых типов применялся специальный макрос `ENUM_DECL(EnumName)` из `sbis-lib/defines.hpp`. В данный момент времени в его применении в новом коде нет никакой необходимости.

Элементы перечислимого типа именуются в верхнем регистре, слова в имени разделяются подчёркиванием. Например: `CONNECTION_INITIALIZATION_FAILED`.

```

1 enum class Foo
2 {
3     ONE = 0,
4     TWO = 1
5 };

```

Сравнение переменной со значением

При сравнении сначала указывается переменная, потом значение.

```

1 // Правильно
2 if( filter.mDirection == common::QueryDirection::TO_OLDER )

```

```

3  {
4      // ...
5  }
6  if( filter.mDirection == common::QueryDirection::TO_NEWER )
7  {
8      // ...
9  }

1 // Неправильно
2 if( common::QueryDirection::TO_OLDER == filter.mDirection )
3 {
4     // ...
5 }
6 if( common::QueryDirection::TO_NEWER == filter.mDirection )
7 {
8     // ...
9 }

```

Комментарии

1. В начале каждого файла следует указать его назначение.
2. В файле с исходным кодом должны быть прокомментированы все классы, методы, флаги и другие типы данных.
3. Для вынужденных решений, когда код осознанно написан "плохо", следует добавить ссылку на Задачу/Ошибку, после решения которой код можно улучшить. В результате разработчик может оперативно понять причину и отследить решение вопроса.

Однострочный комментарий

1. Начинается с двойного символа "косая черта" (//), после которого ставится один пробел.
2. Расположен в отдельной строке над кодом, который комментируется. Благодаря этому упрощается проверка изменений файла на [GitLab](#): заметны изменения в комментариях и исходном коде.
3. Конец комментария обозначается точкой.
4. Имеет такой же структурный отступ, как и сам код.

```

1 // Проверяем, что процессу корректно передаются параметры
2 BOOST_AUTO_TEST_CASE( ProcessUnixArgsAndEnvironTest )

```

Многострочный комментарий

1. Для оформления используйте синтаксис [doxygen](#).
2. Расположен над кодом, который комментируется.
3. Имеет такой же структурный отступ, как и сам код.

```

1 /**
2  * \brief Класс для работы с записью БД, данные полей записи хранятся
   * единым блоком.
3  */
4 class Record

```

Стиль программирования

Общие рекомендации по стилю программирования

1. Если что-то можно описать без программирования, то оно должно быть описано в метаданных приложения (ресурсах, словарях и т. д.).
2. Если данная функциональность уже есть в используемых библиотеках (STL, Boost и тд), то не надо писать свое только потому, что не разобрались с библиотечным!
3. **Нельзя использовать goto!**
4. Нельзя использовать ассемблерные вставки.
5. Глобальные переменные – это не хорошо. Статические поля класса уже лучше, но если есть возможность, лучше обойтись и без них.
6. В коде не должно быть загадочных чисел типа: **10000000L**. Определяйте константы!
7. Если две константы зависят одна от другой, то так и пишите при их определении.
8. Если функция принимает указатель или ссылку на нечто, что не должно этой функцией изменяться, то не забывайте указывать `const`. Особенно это касается параметров `char*`. Например, должно быть так:

```
1 // Плохо
2 View* LoadView( char* name );

1 // Хорошо
2 View* LoadView( char const* name );
```

9. Если метод класса не изменяет поля класса, то указывайте `const` после описания такого метода. Например: `int Count() const`. Это позволит работать с константным указателем на экземпляр данного класса.

10. Выражения.

```
1 // Плохо
2 if( a < b )
3     x = a;
4 else
5     x = b;
6 if( a < b )
7     Func( a );
8 else
9     Func( b );

1 // Хорошо
2 x = a < b ? a : b;
3 Func( a < b ? a : b );
```

11. Нельзя использовать `define` для определения констант. Нельзя делать `define`, когда лень писать конструкции типа `(Reporter*)data`. Вообще, лучше `define` не использовать.
12. В большинстве случаев экземпляры классов в функцию нужно передавать по ссылке.
13. Не ленитесь создавать новые `h`-файлы! Не впихивайте в `h`-файл описания, не очень-то вписывающиеся в него по логике.
14. Если класс является производным, то желательно заводить синоним (`typedef`) `MyParent` для родительского класса и во всех методах обращаться к родительскому через данный синоним.
15. Члены данных класса должны быть сделаны закрытыми (`private`). Это помогает избежать ситуаций, при которых разработчик производного класса нарушает логическую целостность базового класса. Кроме того, это повышает инкапсуляцию, например, в любой момент можно поменять внутреннее представление данных. Для доступа к таким закрытым данным можно сделать пару методов типа `Value` / `SetValue` / `IsState`.
16. Если вам не хватает какой-то базовой функциональности, не спешите её сразу писать, попробуйте сначала найти подходящий класс/функцию в библиотеке. Не нужно изобретать велосипед! Если не смогли найти сами, спросите коллег.
17. Не допускается сбрасывать код с `warning`’ами!
18. Используйте там, где необходимо проверить условие **во время компиляции**, утверждение `static_assert`, тогда если утверждение не выполняется, компилятор сообщит Вам об этом.

```
1 void SomeFunc( int x )
2 {
3     static_assert( sizeof( int ) >= 2, "Не выполнилось мое
```

```

    утверждение!" );
4 }

```

19. Используйте там, где необходимо проверить условие **во время выполнения программы в отладочном режиме**, стандартное утверждение `assert`. Тогда, если утверждение не выполняется во время прогона, программа Вам сообщит об этом. А при компиляции в режиме без отладочного кода, утверждения будут удалены.

```

1 void SomeFunc( int x )
2 {
3     assert( sizeof( int ) >= 2 );
4 }

```

20. Символы указателя и ссылки должны примыкать к названию типа, а не к названию переменной.

```

1 // Плохо
2 char *c;
3 std::string const &str;
4
5 // Хорошо
6 char* c;
7 std::string const& str;

```

Использование пространств имен

Пространства имен важны, т.к. они позволяют структурировать код и избегать конфликтов имен при использовании различных библиотек. В СБИС мы рекомендуем весь код, который не экспортируется, объявлять в подпространстве имен компонента библиотеки, называя его всегда `impl` (`implementation` – реализация). Функциональные куски, связанные между собой, должны входить в одно пространство имен. Все модули СБИС охватываются единым пространством имен `sbis`. Остальные модули входят в подпространства `sbis`.

```

1 // Файл экспортируемого интерфейса IDatabase
2 namespace sbis
3 {
4     namespace dbi
5     {
6
7         // Определяем интерфейс IDatabase
8         class IDatabase
9         {
10             public:
11             };
12
13     }// namespace dbi
14 }// namespace sbis
15
16 // Файл реализации интерфейса IDatabase
17 namespace sbis
18 {
19     namespace dbi
20     {
21         namespace impl
22         {
23             // DataBase реализует IDatabase его конечные пользователи не
                должны видеть
24             class DataBase: public IDatabase
25             {
26             }
27         }// namespace impl
28     }// namespace dbi
29 }// namespace sbis

```


Нельзя использовать конструкции "using namespace boost;", "using namespace std;", и т.д. в hpp-файлах. Например, так писать нельзя:

```
1 vector a;  
2 ptr_vector b;
```

Нужно писать так:

```
1 std::vector a;  
2 boost::ptr_vector b;
```

В файлах .cpp подобные конструкции использовать можно, но только внутри функций и только, если это действительно необходимо.

Ограничения на использование библиотек BOOST

Некоторые библиотеки BOOST приводят к распуханию бинарников и колоссальному замедлению сборки, и в отдельных случаях — к увеличению времени выполнения кода. В связи с этим запущен процесс отказа от ненужных или дублирующихся библиотек.

Ниже представлен список библиотек, запрещенных к использованию, с описанием предлагаемой альтернативы:

boost::bind / std::bind

Вместо boost::bind и std::bind используем [лямбда-выражения C++](#).

Единственный случай, когда можно использовать std::bind — эмуляция захвата переменных в лямбду через move (будет C++14 — избавимся и от этого использования).

boost::lexical_cast

Вместо boost::lexical_cast используем sbis::ToString / sbis::FromString.

Преобразование из строки в некоторый тип:

```
1 try  
2 {  
3     int x = boost::lexical_cast<int>( str );  
4 }  
5 catch( const boost::bad_lexical_cast& )  
6 {}
```

заменяем на

```
1 try  
2 {  
3     int x = FromString<int>( str );  
4 }  
5 catch( const DeStringifyError& )  
6 {}
```

Преобразование некоторого типа в строку:

```
String s = boost::lexical_cast<String>( val );
```

заменяем на:

```
String s = ToString( val );
```

Для работы с "узкими" строками так же нужно использовать ToString / FromString.

boost::format

Вместо boost::format используем sbis::Format.

boost::format является тяжеловесной header-only библиотекой. Поэтому он очень сильно раздувает размер бинарных файлов и сильно замедляет сборку.

sbis::Format имеет минимально возможную header-часть, из-за чего он лишен данных недостатков.

Преимущества sbis::Format:

- Не раздувает размер бинарников.
- Работает через ToString.
- Сам предсказывает и резервирует размер выходного буфера.
- Умеет дописывать в конец String-подобного контейнера.
- Быстро работает.



sbis::Format проектировался как легковесная замена boost::format в большинстве случаев. Он генерирует существенно меньше бинарного кода, меньше отладочных символов, он легче в рантайме.

НО sbis::Format не поддерживает сложные форматы с указанием ширины и т.п (посчитали поддерживать их нецелесообразным, так как в коде всех продуктов Тензора нашлось не более 10 случаев их использования). Поддерживаются только "форматы с %1% такими плейсхолдерами для аргументов".

Ограничения sbis::Format:

- Поддерживает только один формат плейсхолдеров:

```
Format( L"Вот %1% такой"_sv, param )
```

- Строка формата может быть StringView или rvalue String-подобный контейнер.

Пример:

```
String message = ( boost::wformat( L"Ошибка создания документа \"%1%\". Не удалось прикрепить вложение \"%2%\". Код ошибки: %3\" % doc_name % attach_name % err_code ).str();
```

Заменяем на:

```
String message = Format( L"Ошибка создания документа \"%1%\". Не удалось прикрепить вложение \"%2%\". Код ошибки: %3\"_sv, doc_name, attach_name, err_code );
```

boost::assign

boost::assign — пережиток прошлого, оставшийся со времен, когда наши компиляторы не поддерживали initializer_list. Он является "жирной" header-only библиотекой, который ко всему прочему, продуцирует еще и не самый эффективный код. Сейчас нужно использовать [списки инициализации C++11](#).

Пример:

```
std::vector<int> x = boost::assign::list_of( 1 )( 2 )( 3 );
```

Заменяем на:

```
std::vector<int> x = { 1, 2, 3 };
```

boost::range

boost::range — по сути, не нужен. Он является синтаксическим сахаром, который позволяет сэкономить пару букв в коде и запутать незнакомого с этой библиотекой программиста. Веским аргументом против этой библиотеки является медленное препроцессирование её заголовочных файлов, на препроцессирование заголовочников из boost::range компилятор, запущенный на core i7m, тратит более 0.5 с.

boost::atomic

boost::atomic — используем std::atomic. Все используемые нами компиляторы поддерживают std::atomic. Поэтому нет смысла добавлять в приложения дублирующий STL функционал.

Автоматическое форматирование кода по стандарту разработки



Ограничения

Предложенный способ не позволяет переименовывать поля/переменные/классы/функции. Используется только для форматирования отступов, пробелов или длинных строк.

Для Windows

Для установки зависимостей скачайте и установите 64х-битную версию LLVM с [официального сайта](#).

Настройка Qt Creator:

1. Перейдите в **Справка -> О модулях (Help -> About Plugins)**.
2. Активируйте флаги у двух плагинов Beautifier и ClangFormat. Нажмите "Заккрыть" ("Close").
3. Перезапустите Qt и проверьте загрузку плагинов в том же меню.
4. Перейдите в **Инструменты -> Параметры -> Стилизатор -> Clang Format (Edit -> Preferences -> Beautifier -> Clang Format)**. Выберите флаг "Использовать особый стиль" (Use customized style) и добавьте новый с названием Tensor. В **Значение** вставьте настройки стиля из файла [.clang-format](#).
5. Перейдите на вкладку **Инструменты -> Параметры -> Стилизатор -> Основное (Edit -> Preferences -> Beautifier -> Main)**. В меню **Утилита** выберите **ClangFormat**, отключите флаг "Только для файлов текущего проекта", если необходимо. Нажмите "Применить" ("Apply").
6. Проверьте, что выполненные настройки работают в любом плюсовом файле, добавив переносов, где не следует и нажав Ctrl+S.

Для Linux

Вам потребуется утилита clang-format, входящая в состав пакета clang. По умолчанию в репозиториях CentOS 7 присутствует только Clang версии 3.9, которая не совместима для работы с файлом стиля (о нем будет написано далее). Поэтому сначала потребуется установить Clang более старшей версии — 4.0.1.

1. Установите Clang 4.0.1, выполнив в Терминале следующие команды:

```
1 | sudo yum install centos-release-scl
2 | sudo yum install llvm-toolset-7scl enable llvm-toolset-7 bash
```

2. Разместить в корне вашего репозитория [файл](#) со стилями для clang-format.
3. Чтобы отформатировать код, выполните в Терминале команду:

```
1 | clang-format -i -style=file <путь_к_файлу_или_файлам>
```

Благодаря опции style=file утилита clang-format самостоятельно найдет, подложенный в корень репозитория файл. Подробнее о параметрах запуска clang-format можно узнать, выполнив в Терминале команду:

```
clang-format -help
```

Если требуется создать/отредактировать файл стиля, ознакомьтесь с полным [перечнем опций](#).

pre-commit hook

Внедрение pre-commit hook в репозиторий

Добавьте репозиторий [pre-commit-hook](#) как подмодуль в корень репозитория. Название добавляемого репозитория не меняется.

Установка хука

1. Инициализируйте подмодуль.

```
git submodule init
```

2. Извлеките изменения репозитория.

```
git submodule update pre-commit-hook
```

3. В модуле находятся два скрипта - setup-nix и setup-win. Запустите setup-win, если вы используете хук на Windows. На macOS и Linux запускайте setup-nix.

Как работает хук

Хук проверяет измененный код на соответствие стандарту разработки, если код не соответствует, отклоняет коммит и автоформатирует код.

Вносим изменения, делаем коммит и, если код не соответствует clang-format'у, коммит не будет совершен. В этом случае появится сообщение:

```
clang-format.....Failed
- hook id: clang-format
- files were modified by this hook

changed files:
  services/sbis-full-text-search-indexer/implementation/align_hierarchy.cpp

The command "/usr/bin/git commit -F /tmp/QtCreator.pXMdoo" terminated with exit code 1. Cannot commit 1 files
```

Если нужно отключить форматирование кода, оберните его комментариями `// clang-format off` и `// clang-format on`.

Пример:

```
// clang-format off

void Foo(int a) {
}

// clang-format on
```

Код не проверяется хуками:

```
[WARNING] Unstaged files detected.
[INFO] Stashing unstaged files to /home/local/REGION/aa.tretyakova/.cache/pre-commit/patch1743085042-18894.
clang-format.....Passed
[INFO] Restored changes from /home/local/REGION/aa.tretyakova/.cache/pre-commit/patch1743085042-18894.
Фиксирован 1 файл.
```

pre-recieve hook

1. Получите права на репозиторий [git-hooks](#) у [А. Меряшина](#).
2. Создайте свою ветку.

3. В скрипте хука [check_clang-format.py](#) добавьте в словарь REPS название репозитория и версию, с которой нужно внедрить хук.

```
1 REPS = {  
2     'sbis/middleware' : '25.3100'  
3 }
```

4. Создайте Merge request вашей ветки в master.
5. Создайте поручение ([пример поручения](#)) со ссылкой на MR в [отдел мониторинга](#).

Критерии оценки качества кода

[См. документ.](#)